# Personal Project

## Project Overview

In this project, you will choose a real website from **one of the approved business domains**, analyze the business behind that website, identify its core business needs, and build a small backend minimum viable product (MVP) that implements **one selected business need**.

The goal is not only to write working backend code, but also to show that you can think like a product engineer: understand a real business, identify user and business problems, design a practical API-based solution, explain your technical choices, and demonstrate a deployed backend system.

---

## Project Scenario

You must choose a real website from one of the following domains:

- **Financial services**
- **Retail and e-commerce**
- **Customer support operations**
- **Travel and hospitality**

Your chosen website, business domain, and selected business problem must be approved by either **Will**, **Josie**, or **Cate**. **Send your proposal through Slack private messages** before you begin implementation.

You should analyze the website to understand what the business does, who its users are, what services or features it provides, and what problems it is trying to solve.

After analyzing the website, you must identify several possible business needs. Then, you will choose **one business need** and implement a small backend API solution for it.

Your project should represent a small but meaningful portion of a real business backend system. You do not need to build the entire website or recreate the company's full system. Focus on one clear business need and solve it well.

---

## Website Analysis Requirements

Before implementation, you should analyze the website you selected.

Your analysis should answer questions such as:

- What company or product did you choose?
- Which of the approved business domains does it belong to?
- What does this business do?
- Who are the main users or customers?
- What services, products, or features does the website offer?
- What problems does the business appear to solve?
- What actions do users need to perform on the platform?
- What information does the business need to collect, store, or manage?
- What backend data might be needed to support this business?
- What privacy, security, or access-control concerns might exist?

You should be able to explain the core business needs you identified from the website.

Examples of business needs might include:

- Managing financial transactions or payment records
- Tracking loan or credit applications
- Creating transaction or account alerts
- Managing products, inventory, or orders
- Processing returns or refund requests
- Saving shopping carts, wishlists, or user preferences
- Managing customer support tickets
- Tracking ticket status, priority, and assignment
- Managing customer inquiries or service requests
- Managing hotel or travel reservations
- Handling booking changes or cancellations
- Saving travel preferences or search filters

These are only examples. Your business needs should be based on the actual website and business you choose.

---

# Selected Business Need

After analyzing the website, choose **one business need** to implement.

Your selected business need should be meaningful enough to support **at least 5 business API endpoints**.

You are not expected to build an entire financial platform, e-commerce platform, customer support system, or travel booking system. Instead, choose one focused part of the business and build a small backend solution around that need.

For example, instead of trying to build an entire **financial services platform**, you might choose:

- Transaction categorization
- Loan application tracking
- Fraud alert case management
- Expense approval management
- Saved financial search views

Instead of trying to build an entire **retail or e-commerce platform**, you might choose:

- Shopping cart management
- Order management
- Product review management
- Return and refund request management
- Wishlist management

Instead of trying to build an entire **customer support platform**, you might choose:

- Support ticket management
- Ticket assignment and status tracking
- Customer inquiry management
- Support case escalation
- Customer feedback management

Instead of trying to build an entire **travel or hospitality platform**, you might choose:

- Reservation management
- Booking change and cancellation management
- Guest request management
- Travel itinerary management
- Saved trip or destination management

You may use mock data. If your selected business need depends on data from another system, you may assume that the data already exists and create mock data manually or with AI tools.

---

# Required Technologies

Your backend project must use:

- Python
- FastAPI
- MySQL or PostgreSQL
- SQLAlchemy
- Docker
- AWS EC2
- Authentication using either manual JWT or Auth0 OAuth2

You may use AWS S3 if it fits your project, but S3 is not required.

---

# Required Backend Features

Your project must include:

1. **At Least 5 Distinct Business API Endpoints**
    - These must represent real business actions or data operations related to your selected business need.
    - Auth endpoints, login/register routes, health check routes, and deployment test routes do not count toward the 5 required business endpoints.
2. **Authentication**
    - Your API must include user authentication.
    - A basic authenticated user is enough.
    - Admin users, roles, or RBAC are optional bonus features.
3. **Database Integration**
    - Your project must use MySQL or PostgreSQL.
    - You must use SQLAlchemy for database access.
    - You can optionally use Alembic for database migrations.
4. **API Design**
    - Your API should follow the RESTful API design principles covered during training.
    - When designing your API, consider:
        - What resources does your system manage?
        - What URLs should represent those resources?
        - Which HTTP method is appropriate for each operation?
        - What information should the client send through path parameters, query parameters, or request bodies?
        - What information should the API return in the response?
        - What status codes should be returned for successful and unsuccessful requests?
        - What should error responses look like?
        - Do list endpoints need filtering, searching, sorting, or pagination?
5. **Docker Deployment**
    - Your FastAPI application must be containerized.

    - Your app must be deployed to AWS EC2 using Docker.
    - You may also run your database as a Docker container on the EC2 instance.
6. **API Documentation and Testing**
    - You must be able to demonstrate your API using Swagger UI or Postman.
    - Your API contract should be clear, including request bodies, response bodies, status codes, and error handling.

---

# Optional Bonus Features

The following are not required, but can improve your project quality:

- Role-based access control, such as admin and regular user roles
- Pagination
- Filtering and searching
- File upload
- AWS S3 integration
- Strong error handling
- Detailed Swagger documentation
- Domain-specific privacy, security, or compliance considerations

For example, you might discuss financial data security, payment information protection, customer data privacy, access control for support cases, or protection of traveler and reservation information.

---

# AI Tool Usage

You may use AI coding tools such as Cursor, Claude Code, GitHub Copilot, Codex, or ChatGPT to help with implementation.

However, you are responsible for understanding your project. During your presentation, you should be able to explain:

- What website you analyzed
- What business the website represents
- What core business needs you identified
- Which business need you selected to implement
- Why your selected business need matters
- Why your API design makes sense
- How your database models support the business case
- How authentication is handled

- How the application is deployed
- How your endpoints work

---

# Future Frontend Integration

This project focuses on the **backend only**. You do not need to build a frontend at this stage.

Later, during React Lectures, you will return to this project, build a React frontend for it, and connect the frontend to the FastAPI backend you build now.

Keep this future frontend in mind when designing your API. Consider what data the client will need to send, what data it will need to receive, and how the frontend will handle successful responses and errors.

---

# Presentation Requirements

Date: **Oct 5th at 2 PM ET**
Duration: **30-minute presentation**

Slides are not required. You may present directly from the website you analyzed, your code, Swagger UI, Postman, terminal, database client, or deployed EC2 environment.

Your presentation should cover:

1. **Website and Business Domain**
    - What website did you choose?
    - Which business domain does it belong to?
    - What type of business or organization does the website represent?
2. **Website Analysis**
    - What does the business do?
    - Who are the main users or customers?
    - What services or features does the website provide?
    - What core business needs did you identify from the website?
3. **Selected Business Need**
    - Which one business need did you choose to implement?
    - Why did you choose this need?
    - Why does this need matter to the business or its users?
4. **Proposed Backend Solution**
    - What does your API help the business do?
    - What are the main resources/entities in your system?

    - What are the 5 or more business endpoints?
5. **API Contract**
    - Explain the purpose of each main endpoint.
    - Show request examples.
    - Show response examples.
    - Explain important status codes and error cases.
6. **Authentication**
    - Explain how users authenticate.
    - Explain which endpoints require authentication.
7. **Database Design**
    - Explain your main tables/models.
    - Explain how the data supports your selected business need.
    - Mock data is allowed.
8. **Deployment**
    - Show that your FastAPI application is deployed to AWS EC2 using Docker.
    - Demonstrate the deployed API using Swagger UI or Postman.
9. **Live Demo**
    - Demonstrate the main API flow.
    - Show successful requests and, if possible, errors or edge cases.

---

# Final Deliverable

You do not need to submit anything separately. Your presentation is your final deliverable.

During the presentation, you must be able to demonstrate that:

- You analyzed a real website from one of the approved business domains
- You identified core business needs from that website
- You selected one business need to implement
- Your FastAPI application is deployed on AWS EC2
- Your application is running with Docker
- Your API can be tested through Swagger UI or Postman
- Your backend meets the required project features

---

# Key Reminder

Your goal is to use the website as business inspiration, identify realistic backend needs, and implement one focused backend MVP that solves one meaningful business problem.